

Plotly入門

インタラクティブな可視化

rkskmt

1

静的から動へ

2

Plotlyとは

- Plotlyは **インタラクティブな可視化** を作るライブラリ
 - 拡大・縮小
 - ホバーで値を確認
 - 凡例クリックで表示・非表示
 - 3Dグラフを回転
- matplotlibと同じく、Pythonから使える
- Webページ上で動くHTMLとして出力できる

3

インストール & import

- pipからインストール

```
$ pip install plotly
```

Shell

- importは可視化ツールをあつめたplotly.expressをpxとして読み込むのが一般的

```
1 import plotly.express as px
```

Python

① matplotlibとの違い

- matplotlib：静的な図に強い
- Plotly：Web上で動かせる図に強い
- どちらが上ではなく、目的で使い分ける

4

この資料で扱うこと

- **plotly.express** を使ってグラフを作る
- Pandasの **DataFrame** と連携できることを理解
- 棒グラフ・散布図・折れ線・3D散布図を作る
- 最後に **世界一有名な「動くグラフ」** を手で作る

📌 今回は「描き比べ」の回

- 散布図・折れ線・ヒストグラムは **matplotlibで一度作った図** ばかり
- 同じ図を別の道具で作ると、**道具ごとの得意・不得意** が体感できる

5

まずDataFrameを用意する

- PlotlyはPandasの **DataFrame** と相性がよい
- 「どの列をx軸にするか」「どの列をy軸にするか」を列名で指定するだけ
- まずは以下をVS Codeにコピペ

```
1 import pandas as pd
2
3 df = pd.DataFrame({
4     "名前": ["田中", "佐藤", "鈴木", "高橋", "伊藤"],
5     "学科": ["情報", "機械", "情報", "電気", "情報"],
6     "点数": [82, 75, 91, 68, 57],
7 })
8
9 print(df)
```

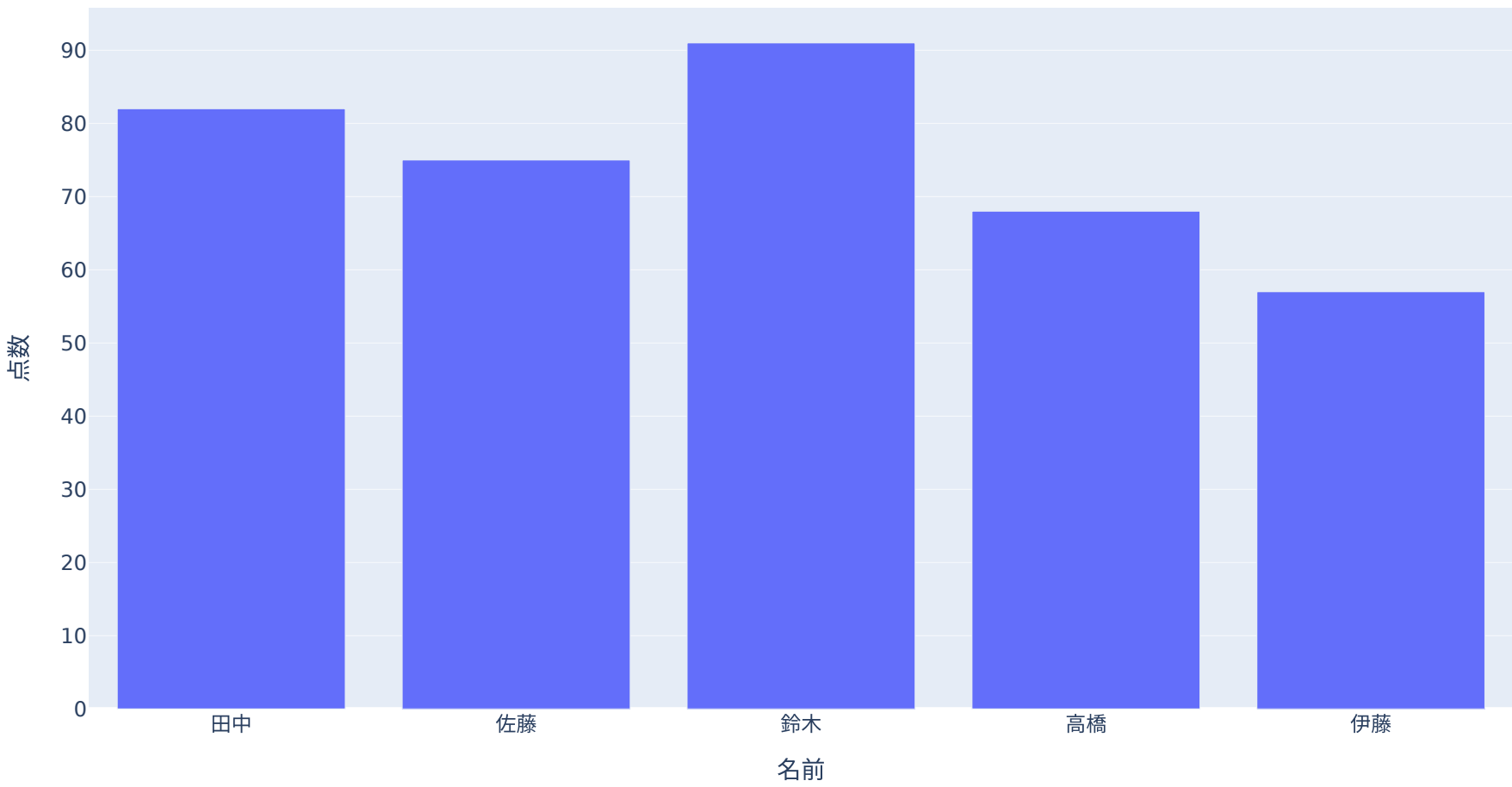
	名前	学科	点数
0	田中	情報	82
1	佐藤	機械	75
2	鈴木	情報	91
3	高橋	電気	68
4	伊藤	情報	57

6

棒グラフを作る

- `px.bar()` で棒グラフを作る
- `x=` と `y=` に列名を指定する
- `fig.show()` で表示する

```
1 import plotly.express as px
2
3 fig = px.bar(df, x="名前", y="点数")
4 fig.show()
```



Plotly Expressの基本形

- `plotly.express` は、短く書くための高レベルAPI
- 基本形はどのグラフでも似ている

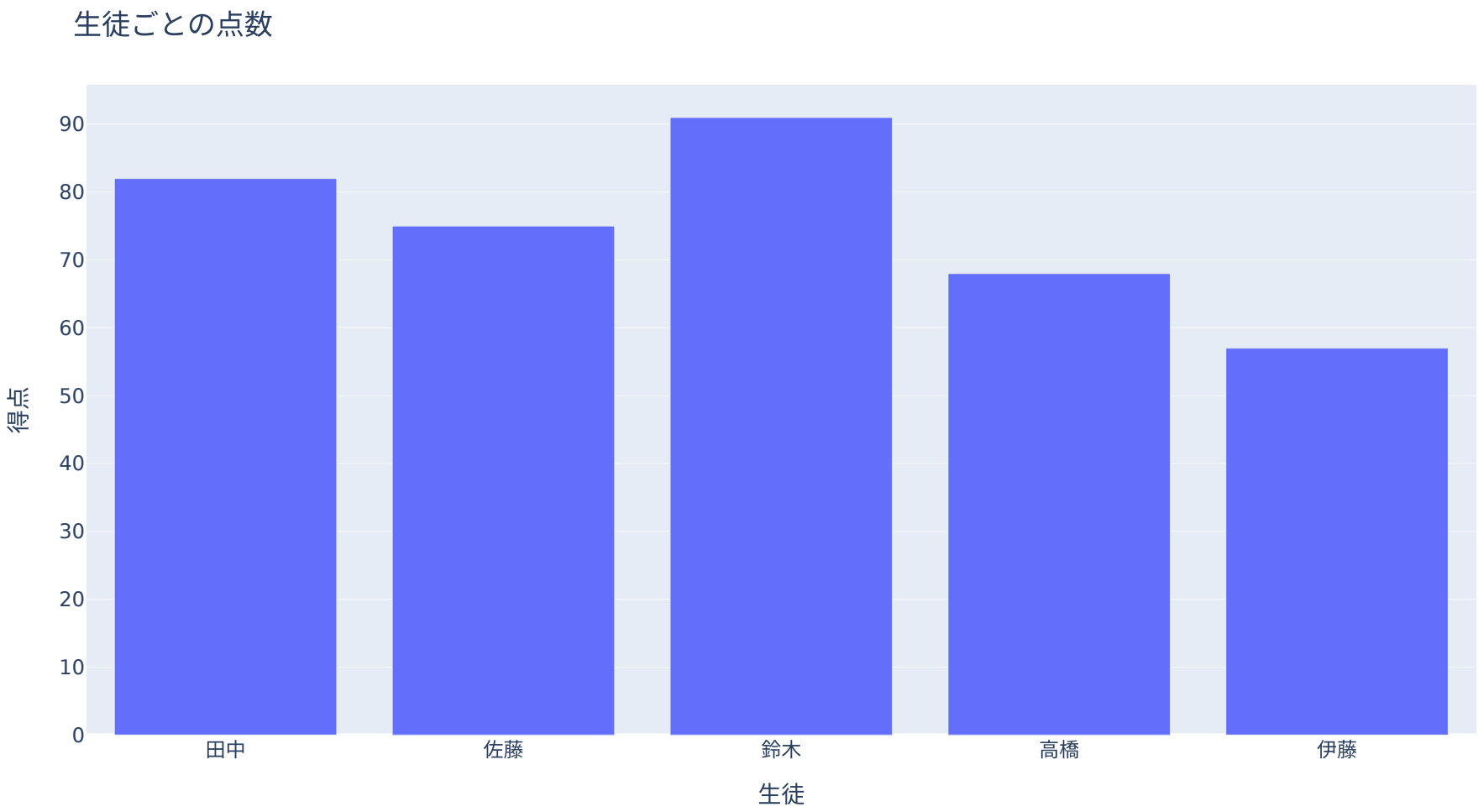
```
1 fig = px.グラフの種類(
2     データフレーム,
3     x="x軸に使う列名",
4     y="y軸に使う列名",
5 )
```

グラフの種類	関数
棒グラフ	<code>px.bar()</code>
散布図	<code>px.scatter()</code>
折れ線	<code>px.line()</code>
ヒストグラム	<code>px.histogram()</code>
3D散布図	<code>px.scatter_3d()</code>

図にラベルとタイトルをつける

- **title=** でタイトル
- **labels=** で軸ラベルや凡例名を変えられる
- **labels** には辞書を渡す

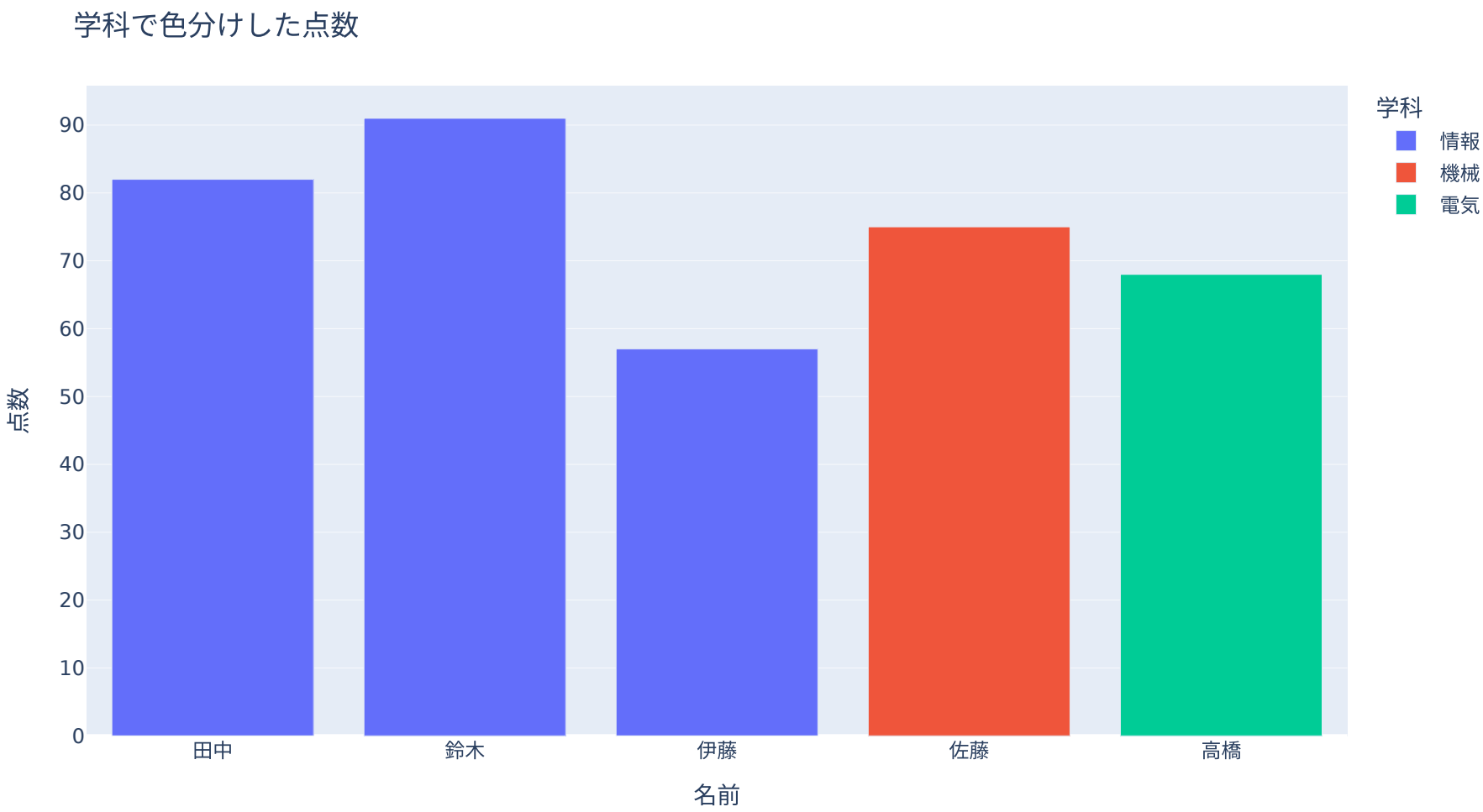
```
1 fig = px.bar(  
2     df,  
3     x="名前",  
4     y="点数",  
5     title="生徒ごとの点数",  
6     labels={"名前": "生徒", "点数": "得点"},  
7 )  
8 fig.show()
```



色分けする

- **color=** に列名を指定すると、その列の値で色分けできる
- カテゴリの違いを見せたいときに便利

```
1 fig = px.bar(  
2     df,  
3     x="名前",  
4     y="点数",  
5     color="学科",  
6     title="学科で色分けした点数",  
7 )  
8 fig.show()
```



💡 何が便利？

- 凡例をクリックすると、特定の学科だけ表示・非表示にできる
- 棒にマウスを乗せると、値が表示される

散布図

- `px.scatter()` で散布図を作る
 - `x` と `y` に数値列を指定する
 - `hover_name=` に列名を指定すると、ホバー時に名前を表示できる
 - データはPandas回と同じ [students.csv](#) (20人)
- csvを **data** フォルダに置いて以下で読み込み

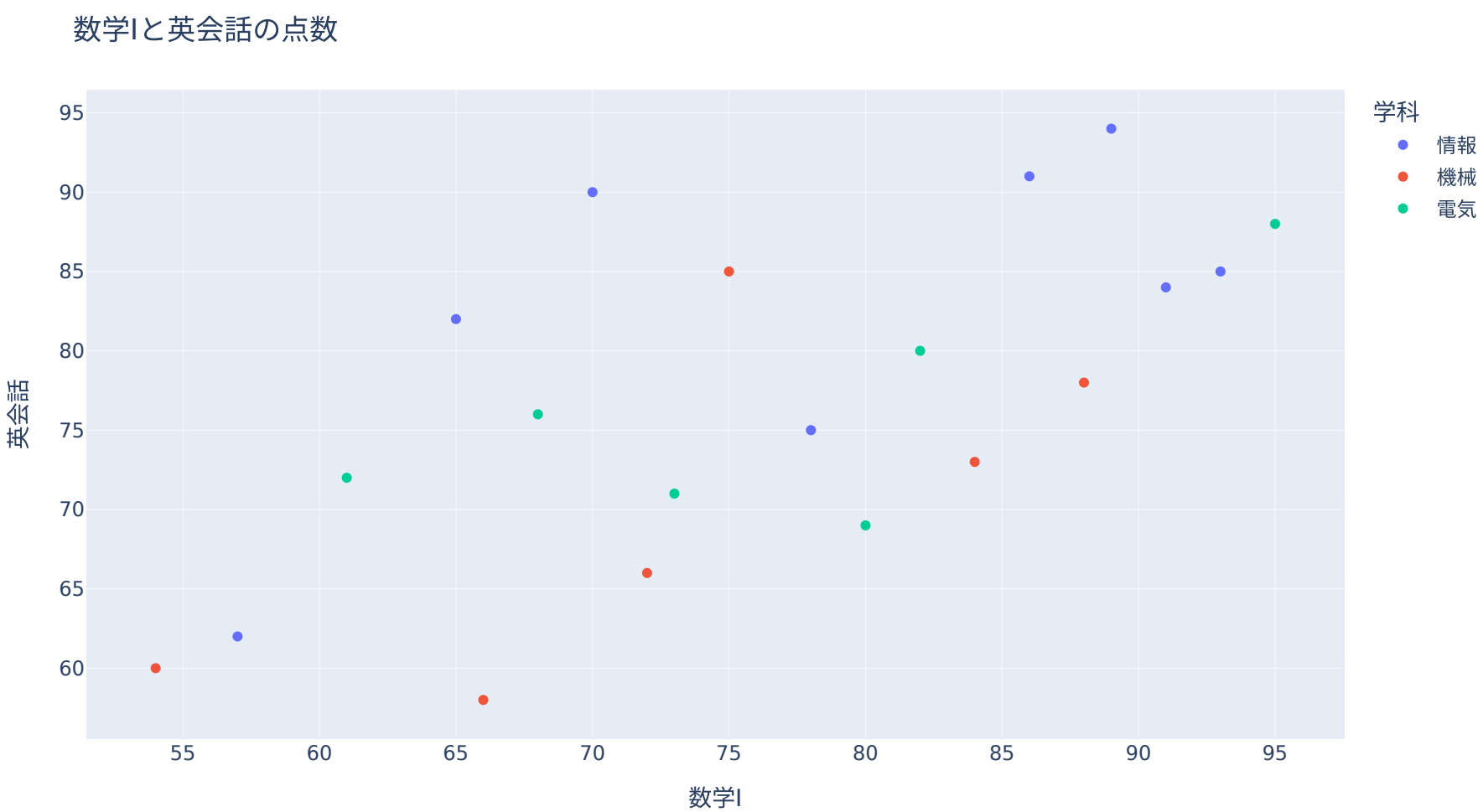
```
1 df_scores = pd.read_csv("data/students.csv")
2 print(df_scores.head())
```

	名前	学科	学年	現代文	数学I	英会話	物理	欠席日数
0	田中	情報	1	80	70	90	85	2
1	佐藤	機械	1	65	75	85	72	4
2	鈴木	情報	2	88	91	84	90	1
3	高橋	電気	2	72	68	76	80	3
4	伊藤	情報	1	58	57	62	65	6

12

散布図のコード

```
1 fig = px.scatter(
2     df_scores,
3     x="数学I",
4     y="英会話",
5     color="学科",
6     hover_name="名前",
7     title="数学Iと英会話の点数",
8 )
9 fig.show()
```



💡 matplotlibとの違いがここに出る

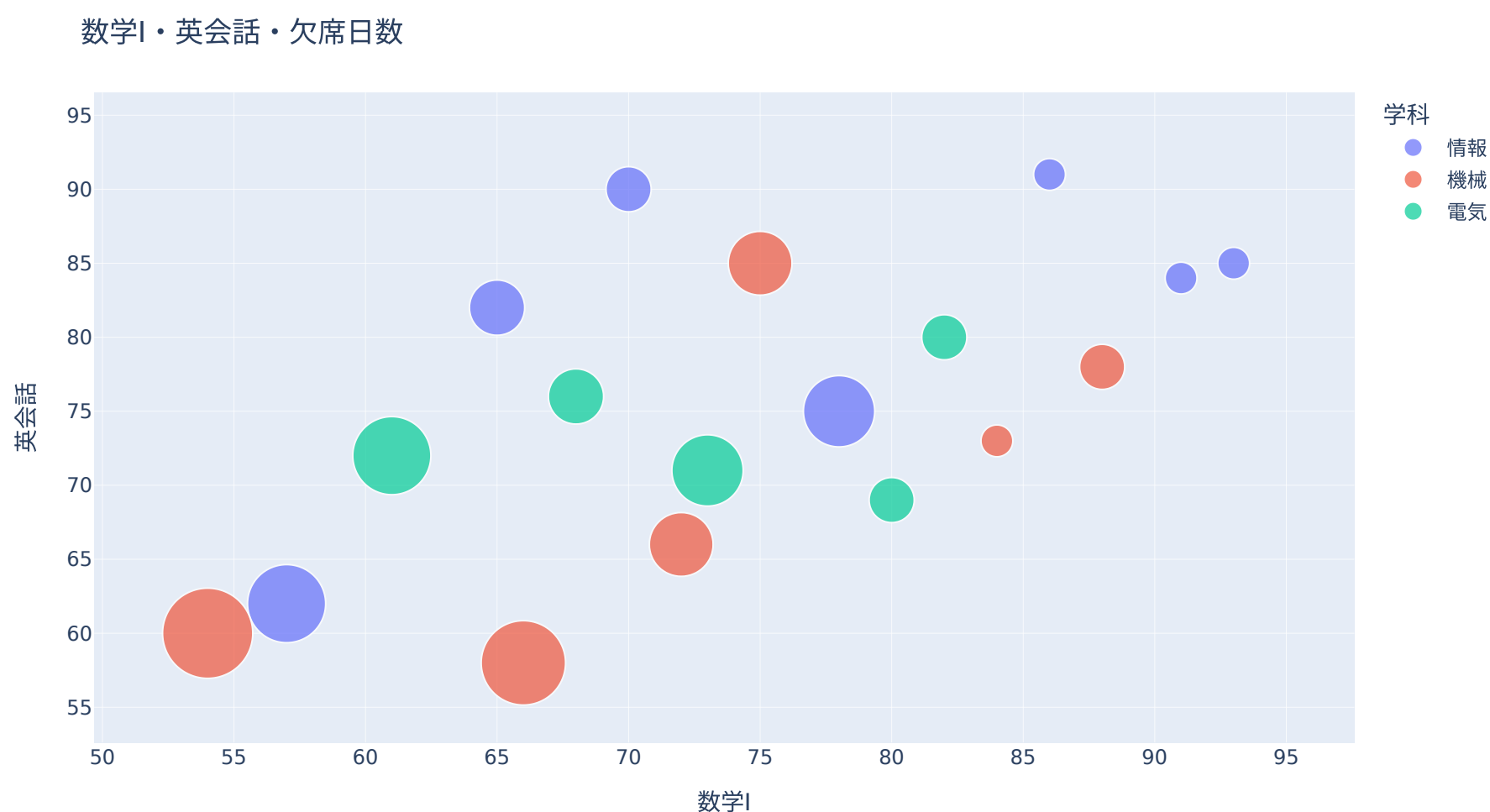
- 外れている点にマウスを乗せると **誰なのか** その場で分かる
- matplotlibなら「この点、誰？」のたびにコードを書き足すことになる

13

点のサイズはデータで可変にできる

- **size=** に数値列を指定すると、点の大きさを変えられる
- 位置（数学I・英会話）に加えて、**欠席日数**という別の量を大きさで重ねられる
- 大きい点ほど欠席が多い — 成績との関係が見えるか？

```
1 fig = px.scatter(  
2     df_scores,  
3     x="数学I",  
4     y="英会話",  
5     color="学科",  
6     size="欠席日数",  
7     hover_name="名前",  
8 )  
9 fig.show()
```



14

折れ線グラフ

- **px.line()** で折れ線グラフを作る
 - 時間変化や順序のあるデータに向いている
 - 題材は可視化編の **東京150年の年平均気温**（気象庁）
- データ：[tokyo_temp_annual.csv](#) （**data** フォルダに置く）

```
1 df_temp = pd.read_csv("data/tokyo_temp_annual.csv")  
2 print(df_temp.head())
```

	year	temp
0	1876	13.6
1	1877	14.2
2	1878	13.8
3	1879	14.6
4	1880	14.1

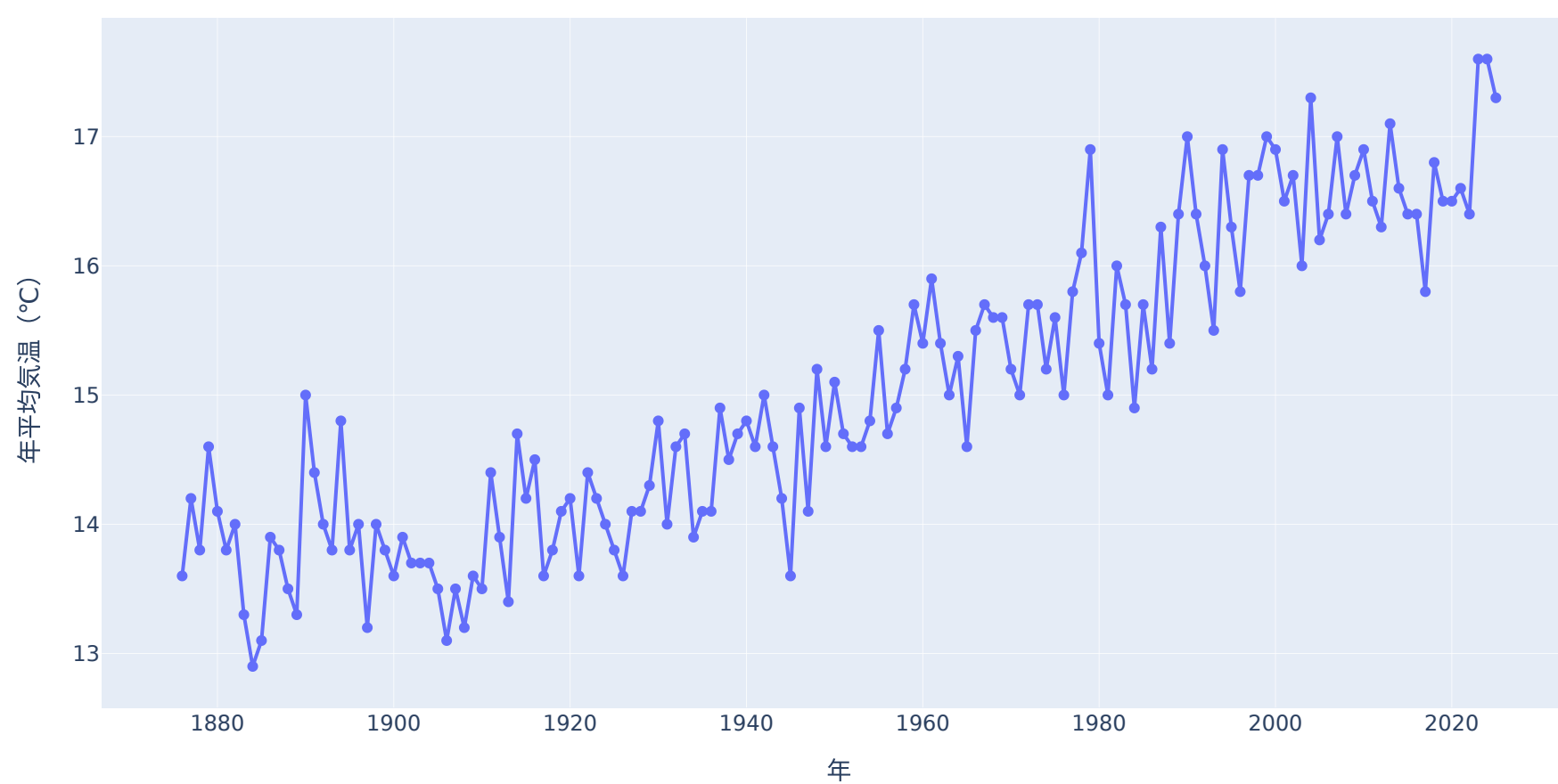
15

折れ線グラフのコード

```
1 fig = px.line(df_temp, x="year", y="temp", markers=True)
2 fig.show()
```

Python

東京の年平均気温（1876～2025）



💡 自分の指でチェリーピックしてみる

- **ドラッグで2006～2017年あたりを選択** してズーム → 「寒冷化」に見える
- **可視化編の騙しグラフ** は、この **切り取り** と同じ操作だった
- **ダブルクリックで全体に戻る** — 全体を確認するクセをつける

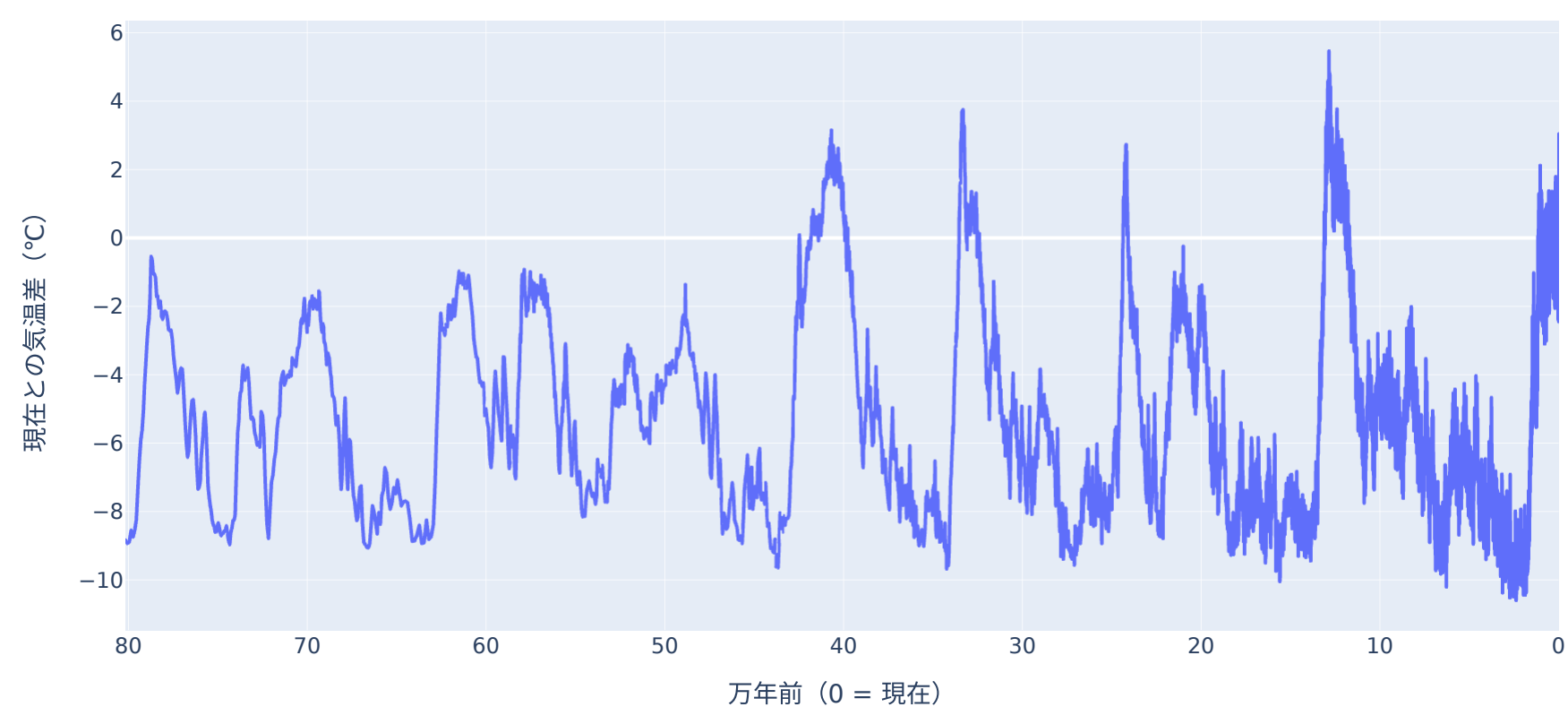
ズームアウト：80万年の気温

- 南極の氷から復元した気温 — **80万年前**まで遡れる（EPICA Dome C）。縦軸 **0°C=現在の水準**（直近1000年の平均）、マイナスほど寒い
- ドラッグでズーム → 氷期と間氷期を **約10°Cも往復**。東京150年の変化は、この巨大なリズムのごく一部
- 同じ「気温」でも **見せるスパンで物語が変わる**
- データ：[epica_temp.csv](#)  ([data](#) フォルダに置く)

```
1 df_temp_long = pd.read_csv("data/epica_temp.csv")
2 df_temp_long["万年前"] = df_temp_long["age_kyr_bp"] / 10 # 千年 → 万年 で読みやすく
3
4 fig = px.line(df_temp_long, x="万年前", y="temp_anomaly_c")
5 fig.show()
```

Python

南極の氷から復元した気温（過去80万年）

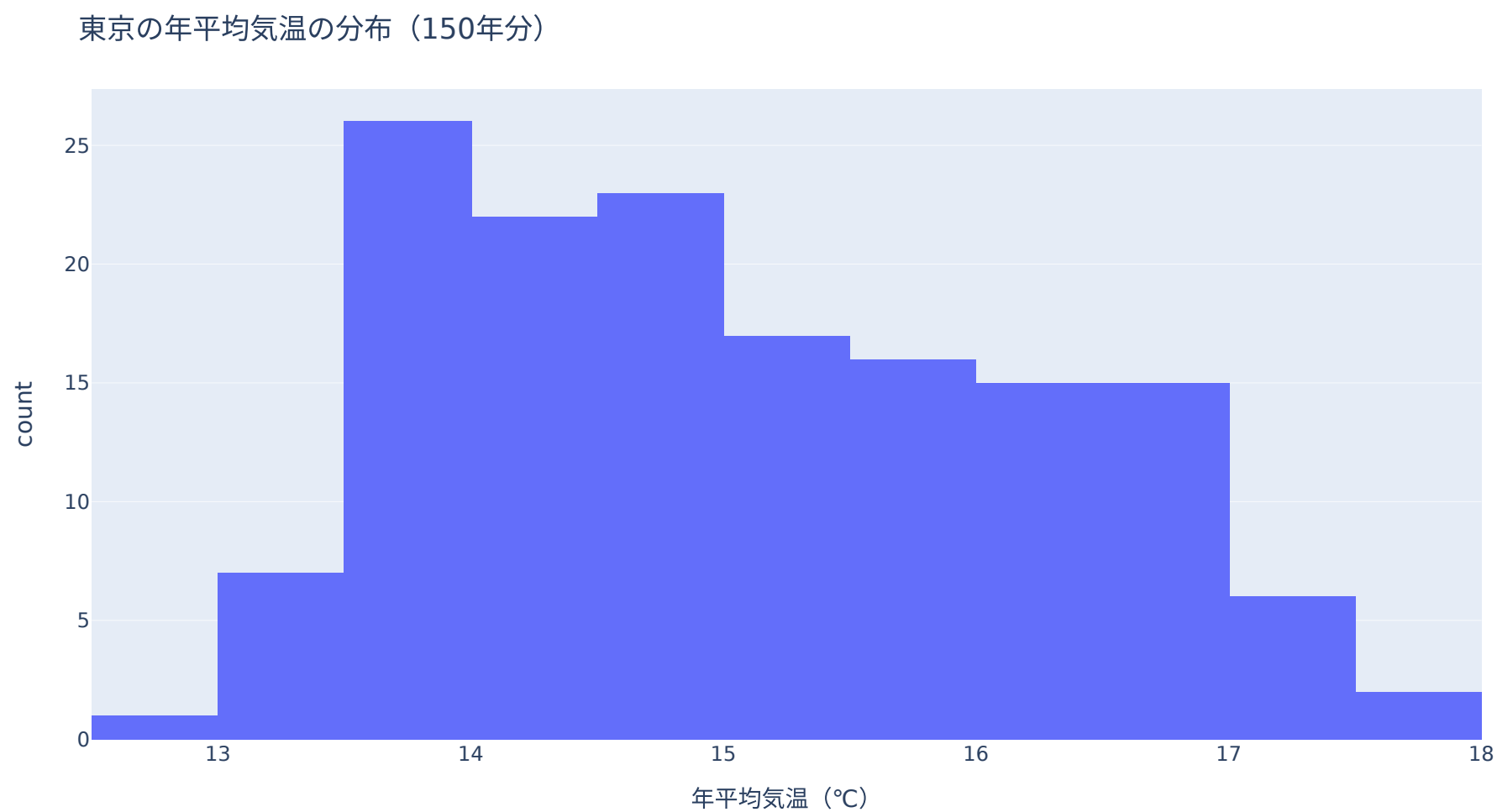


ヒストグラムのコード

- `px.histogram()` で分布を見られる
- `nbins=` で区間の数を指定する
- 同じ150年の気温データ — 折れ線は **変化**、ヒストグラムは **分布** を見せる

```
1 fig = px.histogram(df_temp, x="temp", nbins=20)
2 fig.show()
```

Python



- 棒にマウスを乗せると **どの気温帯が何年分か** まで分かる

18

Pandasの集計結果をPlotlyへ

19

学科ごとの平均点を計算

- Pandasで集計してからPlotlyで可視化する
- Pandas入門と同じ `data/students.csv` を使う
- `groupby()` の結果は **Series**
- Plotlyに渡すため、いったん `reset_index()` でDataFrameに戻す

```
1 csv_path = "data/students.csv"
2 df_students = pd.read_csv(csv_path)
3 subjects = ["現代文", "数学I", "英会話", "物理"]
4
5 df_students["平均点"] = df_students[subjects].mean(axis=1)
6 avg = df_students.groupby("学科")["平均点"].mean()
7 print(avg)
```

Python

```
学科
情報    80.062500
機械    71.625000
電気    77.583333
Name: 平均点, dtype: float64
```

20

SeriesをDataFrameに戻す

- `reset_index()` で、indexを普通の列に戻せる
- Plotly ExpressはDataFrame + 列名指定が書きやすい

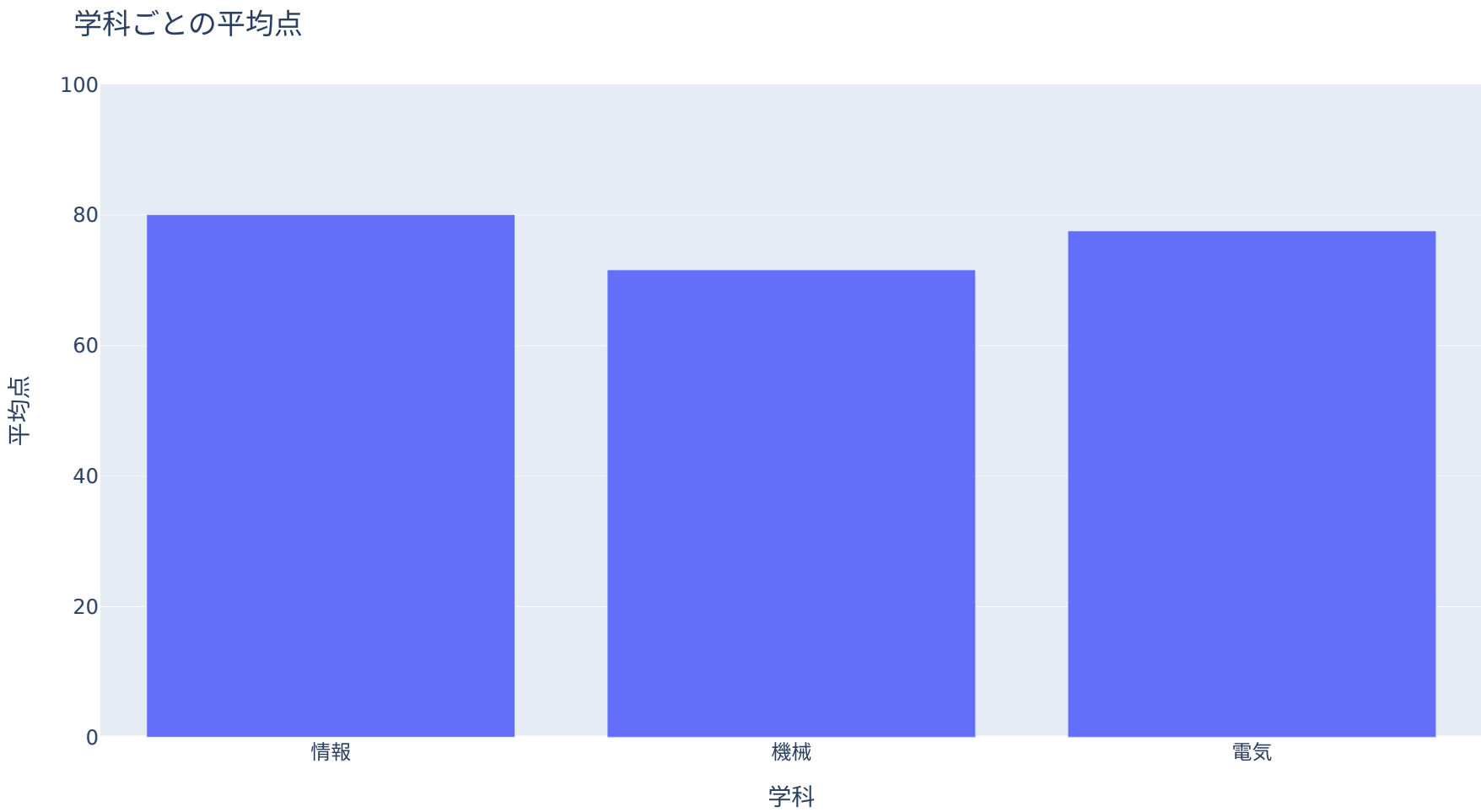
```
1 avg_df = avg.reset_index()  
2 print(avg_df)
```

	学科	平均点
0	情報	80.062500
1	機械	71.625000
2	電気	77.583333

```
1 fig = px.bar(avg_df, x="学科", y="平均点")  
2 fig.show()
```

21

集計結果をPlotlyで可視化



i matplotlibとの違い

- 1 matplotlib: Series → index / values
- 2 Plotly: Series → `reset_index()` → DataFrame

22

そのままDataFrameを作る方法

- **groupby(..., as_index=False)** とすると、最初からDataFrameで結果を受け取れる
- Plotlyで使う場合はこちらも便利

```
1 avg_df = df_students.groupby("学科", as_index=False)["平均点"].mean()  
2 print(avg_df)
```

	学科	平均点
0	情報	80.062500
1	機械	71.625000
2	電気	77.583333

```
1 fig = px.bar(avg_df, x="学科", y="平均点")  
2 fig.show()
```

3Dと書き出し

3D散布図

- **px.scatter_3d()** で3D散布図を作れる
- Web上ではドラッグして回転できる
- 3つの数値列の関係を見るときに使う
- **data/students.csv** の3科目（数学I・英会話・物理）を使う

```
1 df_3d = pd.read_csv("data/students.csv")  
2 print(df_3d.head())
```

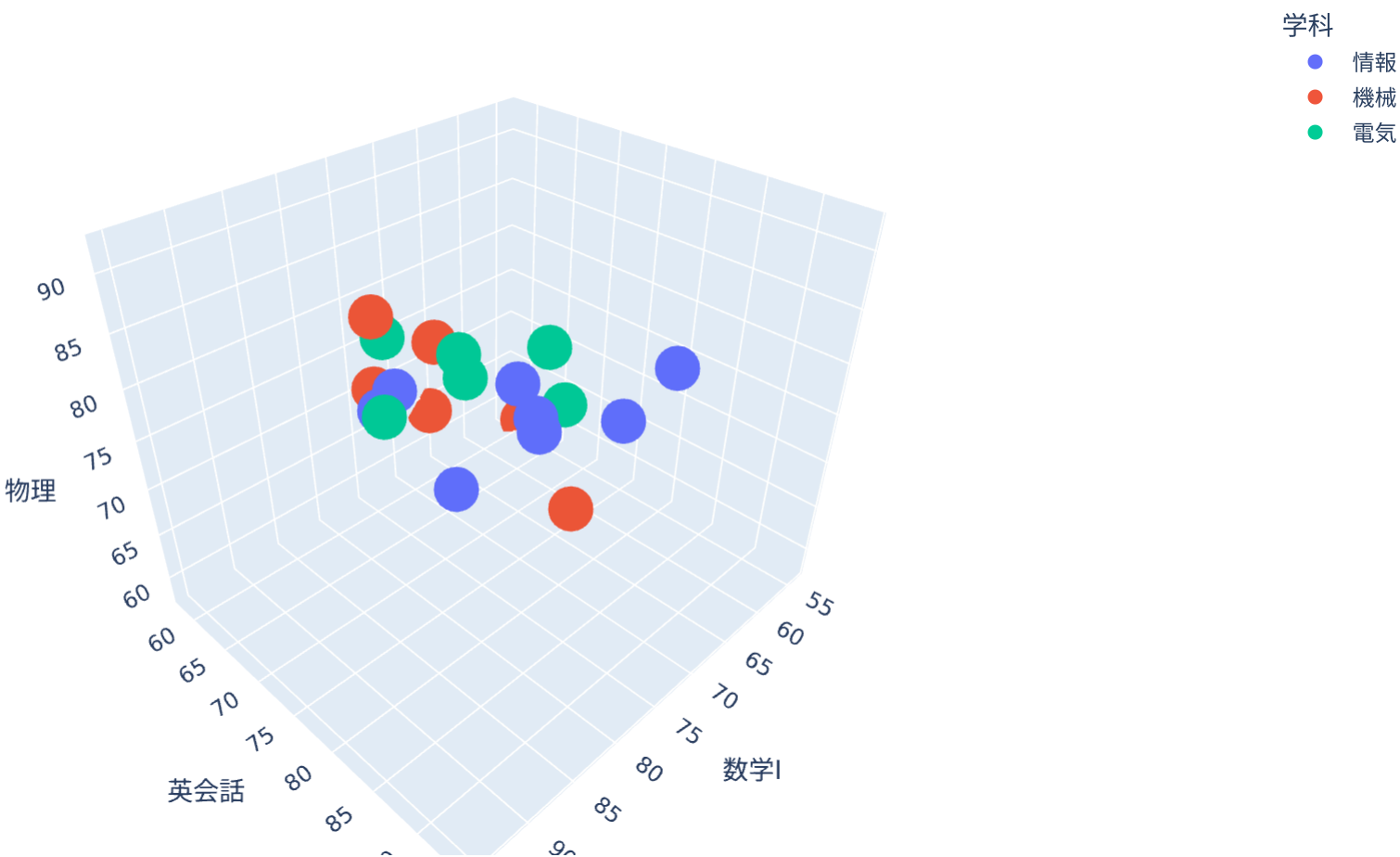
	名前	学科	学年	現代文	数学I	英会話	物理	欠席日数
0	田中	情報	1	80	70	90	85	2
1	佐藤	機械	1	65	75	85	72	4
2	鈴木	情報	2	88	91	84	90	1
3	高橋	電気	2	72	68	76	80	3
4	伊藤	情報	1	58	57	62	65	6

3D散布図を作る

```
1 fig = px.scatter_3d(  
2     df_3d,  
3     x="数学I",  
4     y="英会話",  
5     z="物理",  
6     color="学科",  
7     hover_name="名前",  
8 )  
9 fig.show()
```

Python

3科目の点数



① matplotlibにも3Dはある（参考）

- **ax = fig.add_subplot(projection="3d")** で描けるが、**回せない**
- 紙やスライドの静止画では奥行きが伝わりにくい
- **回せるPlotlyでこそ3Dは生きる** — だから3Dはこの回で扱った

HTMLとして保存する

- Plotlyの図はHTMLとして保存できる
- ブラウザで開けるので、相手に共有しやすい
- 実は本スライドは、このHTMLを以下のようにiframeとして埋め込んでいる

```
1 fig.write_html("graph.html")
```

- 何も指定しないと **Plotly本体ごと** HTMLに埋め込まれる（約4MB）
→ ファイルは重いが、**ネットがなくても動く**

① include_plotlyjs で本体の置き場所を選ぶ

指定	HTMLのサイズ	閲覧時のネット
(指定なし)	重い（約4MB）	不要
"cdn"	軽い	必要
"directory"	軽い（ plotly.min.js を 同じフォルダに書き出す）	不要

本スライドは **"directory"** を使っている — 教室のWi-Fiが切れても動くように

matplotlibとPlotlyの使い分け

目的	向いている道具
レポートに貼る静止画	matplotlib
論文・印刷用の細かい調整	matplotlib
ホバーで値を見たい	Plotly
拡大して細部を見たい	Plotly
Webページで共有したい	Plotly
3Dを回して見たい	Plotly

- まずはmatplotlibで基本を覚える
- Webで動かしたいときにPlotlyを使う

演習：Gapminder “Wealth & Health of Nations”を自分で可視化

Gapminder “Wealth & Health of Nations”

- 可視化編の実例集で見た、世界で一番有名な「動くグラフ」
→ 横軸：1人あたりGDP、縦軸：平均寿命、バブルの大きさ：人口
- Plotlyには **このデータが同梱** されている

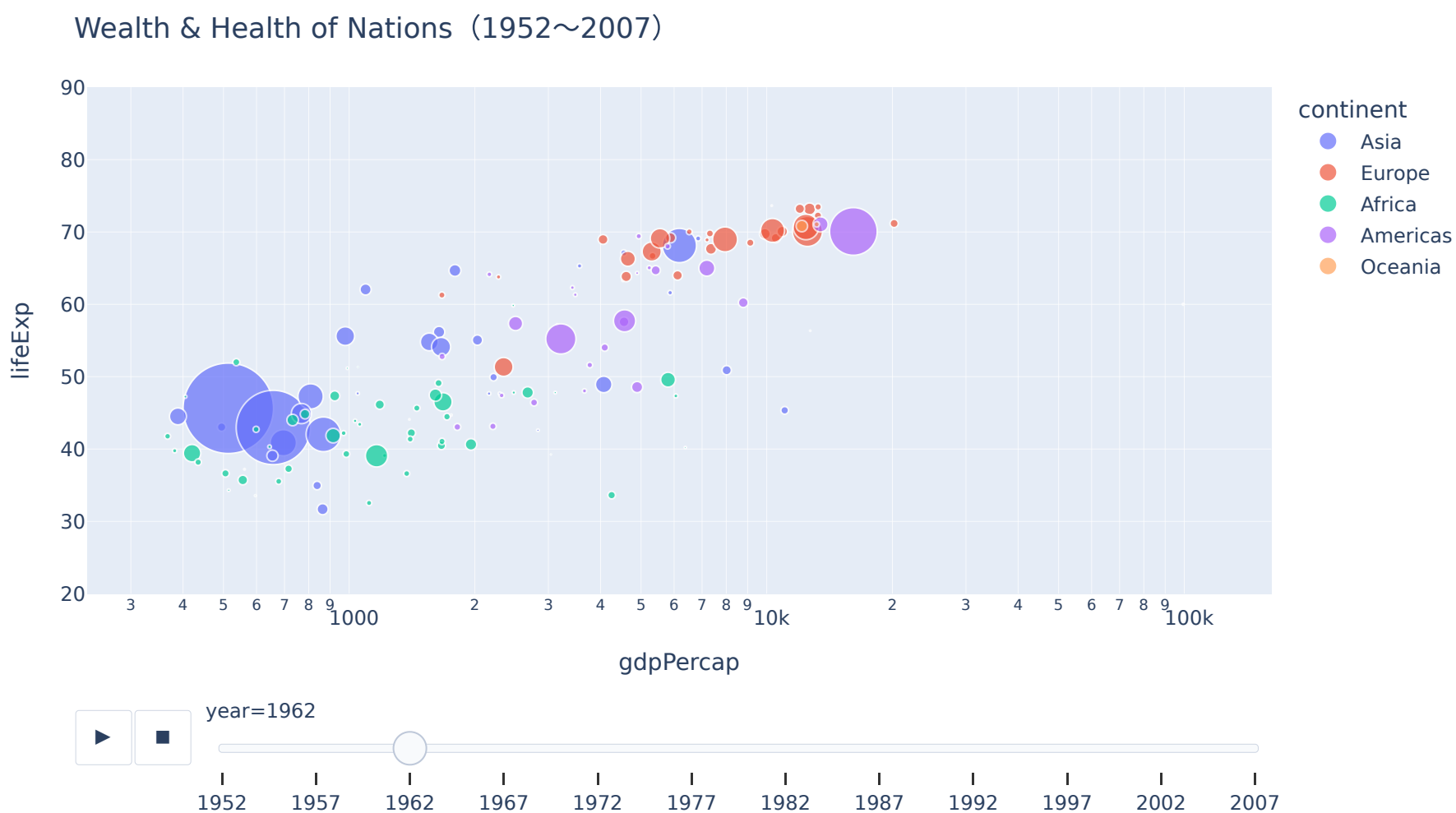
```
1 gap = px.data.gapminder()  
2 print(gap.head())  
3 print(gap.shape)
```

	country	continent	year	lifeExp	pop	gdpPercap	iso_alpha	\
0	Afghanistan	Asia	1952	28.801	8425333	779.445314	AFG	
1	Afghanistan	Asia	1957	30.332	9240934	820.853030	AFG	
2	Afghanistan	Asia	1962	31.997	10267083	853.100710	AFG	
3	Afghanistan	Asia	1967	34.020	11537966	836.197138	AFG	
4	Afghanistan	Asia	1972	36.088	13079460	739.981106	AFG	
	iso_num							
0	4							
1	4							
2	4							
3	4							
4	4							
(1704, 8)								

- **142カ国 × 12時点（1952～2007年）** の実データ

動くバブルチャートを簡単コーディング

```
1 fig = px.scatter(  
2     gap,  
3     x="gdpPercap",  
4     y="lifeExp",  
5     size="pop",  
6     color="continent",  
7     hover_name="country",  
8     log_x=True,  
9     size_max=55,  
10    animation_frame="year",  
11 )  
12 fig.show()
```



- ▶ を押す と55年分の世界が動く（カーソルをあわせると国名がでる）

31

演習：学科ごとの平均点をPlotlyで描く



- **data/students.csv** を読み込む
- **groupby("学科", as_index=False)** で平均点を作る
- **px.bar()** で棒グラフを作る
- **color="学科"** を指定して色分けする
- y軸を0~100にする

▶ 解答例を見る

32

まとめ

- Plotlyは **インタラクティブな可視化** に強い
- **plotly.express** は短いコードで書ける
- 基本形は **px.bar(df, x="列名", y="列名")**
- 色分けは **color=**
- ホバー表示は **hover_name=**
- Pandasの集計結果は、DataFrameにしてからPlotlyに渡すと分かりやすい
- HTMLとして保存して、Webページに埋め込める

① 道具は揃った

- 描く道具（matplotlib / Plotly）と表を扱う道具（Pandas）が揃った
- 次回は **材料**の調達：世界中の人がいま何を見ているかをAPIで取得